Quadrotor Control Project

ECGR 5411 Control Theory I, Fall 2025
November 24, 2025

Curtis Chung
cchung10@charlotte.edu

Amy Edward
aedward@charlotte.edu

Gavin Kelley
gkelley3@charlotte.edu

I. Quadrotor Observability Exercise

For each C_i where $i = 1, 2, 3$:

1.1A. The following output blocks summarize the observability matrix ranks:

```
Suite 1: Rank of observability matrix
         = 12 (out of 12)
Suite 2: Rank of observability matrix
         =  8 (out of 12)
Suite 3: Rank of observability matrix
         = 12 (out of 12)
```

1.1B. Figs. 1, 2, and 3 show the observability Gramian eigenvalues plotted on a logarithmic scale.

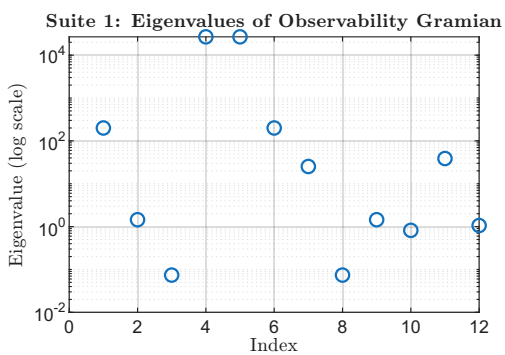

Fig. 1. Full Position + Attitude Sensors (Basic)

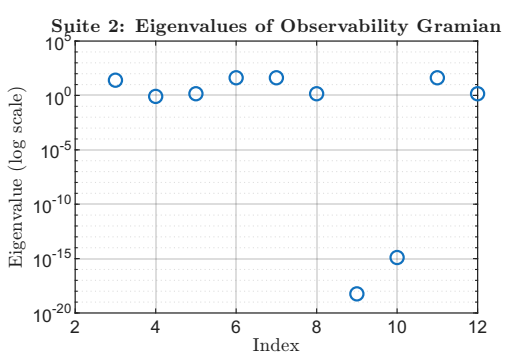

Fig. 2. IMU + Altimeter (Reduced)

1.1C. A small eigenvalue implies that there is a direction in the state space that produces no change in outputs over the time window. Therefore, a small eigenvalue is almost unobservable along that direction, is unable to reconstruct that combination state from the available sensors, and is sensitive to noise.

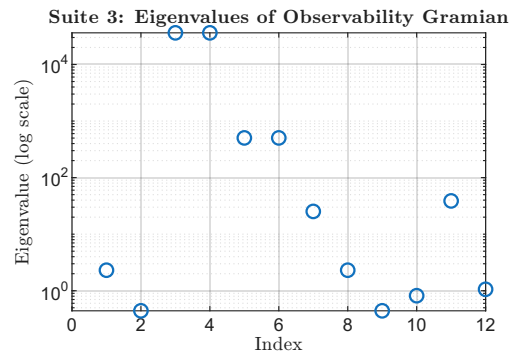

Fig. 3. Mixed & Noisy (Student-Designed)

1.1D. The eigenvalues of the Gramian vary over several orders of magnitude because different sensors in matrix C_i couple to the states with varying strengths, resulting in some states influencing the output more than others. Along with it, the system dynamics in the A matrix evolve on different time scales, making certain states contribute more output energy than others. Together with A and C, it creates a mix of strongly observable and barely visible modes, which leads to a wide spread of eigenvalues. From a numerical viewpoint, this creates an ill-conditioned Gramian, where large and small eigenvalues are together, revealing the differences in how each state is observed.

1.2A. When the rank of the observability matrix is less than 12, some combinations of states do not significantly influence the measurements. For Suite 1, the rank is equal to 12, which indicates full rank; therefore, all states are observable. Suite 2 exhibits a rank of 8 out of 12, where the horizontal position and velocity directions are unobservable, indicated as zeros in the C matrix. The designed observability matrix in Suite 3 has a rank of 12, which is similar to that of Suite 1; therefore, all states are observable.

1.2B. Suites 1 and 3 exhibit full observability of position

and attitude, with a GPS sensor selected in Suite 3 to achieve this. For Suite 2, the horizontal positions and velocities are not directly measured. Adding the horizontal position and horizontal velocity restores observability for suite 2.

1.2C. If the position measurements are removed, it would break the direct link between position and its time derivative; therefore, translational velocities can no longer be corrected using position feedback. This makes the horizontal velocity states unobservable, and velocity estimates could also be inaccurate due to small errors not being detected by the sensors.

1.2D. Yes, time-varying measurements can improve observability. Even if a fixed sensor configuration does not observe all states, allowing the measurement matrix $C(t)$ to change over time can provide different "views" of the system dynamics. The time-varying observability Gramian gathers information from all sensor orientations and positions, and this combined information can span the full 12-state space even when no single $C(t)$ matrix is full-rank.

1.3A. The C_3 matrix is shown below.

$C_3 =$

$$\begin{matrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{matrix}$$

The GPS, optical flow, IMU gyro, and magnetometer sensors were selected to create this matrix.
The GPS provides the quadrotor with its absolute position in the internal frame. Measuring x and y gives a horizontal position that can estimate the horizontal velocity. Measuring altitude z also provides estimates of altitude velocity. The GPS prevents drift in the translational states and motion.
The optical flow sensor measures the horizontal velocity relative to the ground. These directly inform how fast the vehicle is moving sideways/forward; when combined with the GPS x and y measurements, they improve observability for both position and velocity. It can also provide indirect information about roll ϕ and pitch θ, because translational acceleration and velocity depend on the attitude through gravity.
IMU gyros measure the body angular rates, which are time derivatives of the attitude angles. These measurements tightly constrain rotational dynamics, allowing the estimator to estimate the time evolution of the attitude states.
The magnetometer measures the yaw angle ψ by sensing the Earth's magnetic field. This is important because Yaw cannot be inferred from translational motion or gravity, making it essential to have a direct measurement of the yaw angle ψ.
Together, this combination of sensors provides a realistic sensor suite for controlling a quadrotor and offers enough data to make the majority of the modes observable.

1.3B. To demonstrate numerically that C_3 is observable, the following was applied in MATLAB:

```
C3 = zeros(8,12);
% GPS: x, y, z
C3(1,1) = 1;   % x
C3(2,2) = 1;   % y
C3(3,3) = 1;   % z
% Magnetometer: psi
C3(4,6) = 1;   % psi
% Optical flow: x_dot, y_dot
C3(5,7) = 1;   % x_dot
C3(6,8) = 1;   % y_dot
% Gyros: p, q
C3(7,10) = 1;  % p
C3(8,11) = 1;  % q
C = C3;
Ob = obsv(A, C); % observability matrix
r = rank(Ob);
fprintf('Suite 3: Rank of observability
    matrix = %d (out of 12)\n', r);
```

```
Suite 3: Rank of observability matrix
        = 12 (out of 12)
```

For a 12-state system, observability requires a rank = 12; so therefore, the C_3 is observable.

1.3C. For this C_3 matrix, applying `Wd = dlyap(Ad', C'*C);` results in an error, so a similar loop to that of the continuous observability Gramian was constructed. The plot of the discrete Gramian eigenvalues is shown in Fig. 4. Both sets of eigenvalues follow the same overall pattern, increasing across the state indices and spanning many orders of magnitude, which shows that they identify the same strongly and weakly observable modes. The discrete-time eigenvalues are consistently larger in magnitude because the discrete Gramian is formed from a summation rather than an integral, but the relative ordering of the modes remains consistent with the continuous-time case. All eigenvalues are strictly positive, confirming that the system is observable in both continuous and discrete time. The similar shapes of the two curves in the plot indicate that the discrete-time approximation captures the same observability characteristics as the continuous-time system despite the scale difference.

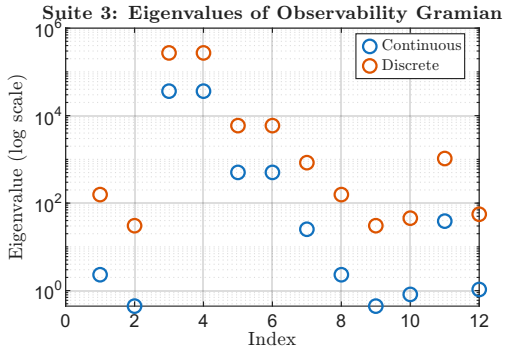

Fig. 4. Eigenvalues for Continuous and Discrete Observability Gramians

II. Continuous and Discrete Quadrotor Control using Receding Horizon Gramian Methods

A. Design and Analysis

The following points in this section discuss the project design process, where significant changes to the MATLAB starter code and additional observations from the simulations are included.

Design 1: The first few points in this section primarily outline the design process before the project hint was released on Saturday, November 22nd.

Initial Overshoot An initial run of the code showed a high overshoot in positions. Making the LQR feedback act on the error helped fix this issue by reducing the steady state error.

LQR Tuning The original LQR gains provided in the code were used initially, but it was difficult to track all waypoints accurately in the given sequence. Gains in both the Q and R matrices were estimated through trial and error. It was observed that increasing the weight on velocity improved velocity tracking but reduced position accuracy, and vise versa. As a new starting point, identity matrices were used for both Q and R, as suggested in the video in [1]. This adjustment allowed the positions to align more closely with the waypoints in the defined sequence, although some velocity errors still remained. From here, the gains were adjusted to meet all positional waypoints.

Parameter Tuning With the LQR gains fixed, other parameters such as waypoint hold, planning horizon, replan interval, and integration step were adjusted to minimize velocity errors. The resulting velocities were generally in the correct direction, but their magnitudes were incorrect. This process provided additional insight into how each parameter influences the behavior of the simulation.

1) Design 2: This part emphasizes the changes made to acquire the final design.

Non-LQR Tuning With LQR disabled and the velocities set to zero, the quadrotor successfully reached all specified waypoints. Introducing non-zero velocities to the targets caused deviations from the desired behavior. Adjusting the roll, pitch, and yaw angles (ϕ, θ, ψ) along with their rates (p, q, r) yielded only slight improvements in error, and tuning one waypoint often degraded performance at others.

Final Design The horizon time, replan interval, waypoint hold, and Gramian regularization parameters were adjusted until the design reached all position and velocity waypoints. The horizon time and replan intervals were chosen to have a smaller overlap between them, with the waypoint hold set at 20 s. The regularization was decreased to meet all specifications for all 6 cases. Since the design met all specifications, the LQR feedback was set to a small fraction of what it was in Design 1.

Differences between Initial and Final Design Settling time performed worse when all the waypoints were reached in Design 2. This is likely due to the weak LQR feedback, the long waypoint hold, and the long replan interval, all of which could cause slower convergence. This can be seen in the position vs. time plots (Figs. 6, 10, 14, 17, 20, and 23), where the final design behaves more like a sine wave than the earlier, square-wave-like states. These square-wave states from Design 1 are shown in Fig. 5 to demonstrate the successful settling time.

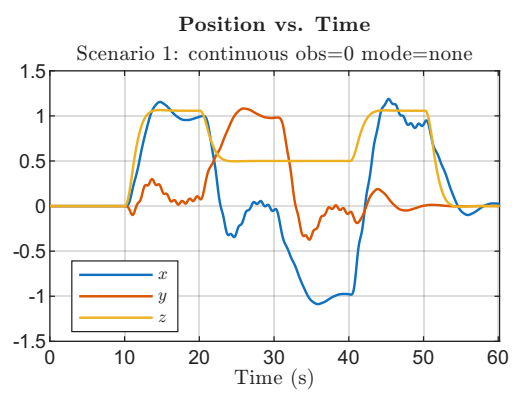

Fig. 5. Design 1, Scenario 1: Position vs Time (Position Tracking, Doesn't Meet Velocity Waypoints, Moderate LQR Feedback)

B. Results

The position, velocity, motor speed, and requested analysis for each scenario are presented in this section. Additionally, Scenarios 1 and 2 feature a 3D trajectory plot (Figs. 9 and 13) to visualize the quadrotor successfully reaching each target. An overlayed view of all 6 Scenario trajectories is shown in Fig. 31. Finally, a tabulated view of all Scenario position and tracking metrics is shown in Table I.

Scenario 1: Continuous-Time Receding Horizon Planning: Figs. 6, 7, and 8 show the results for the continuous-time receding horizon quadrotor.

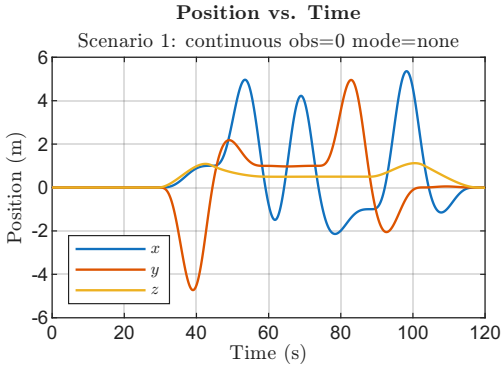

Fig. 6. Scenario 1 Position vs. Time

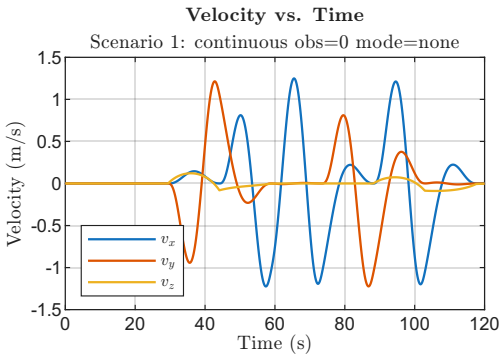

Fig. 7. Scenario 1 Velocity vs. Time

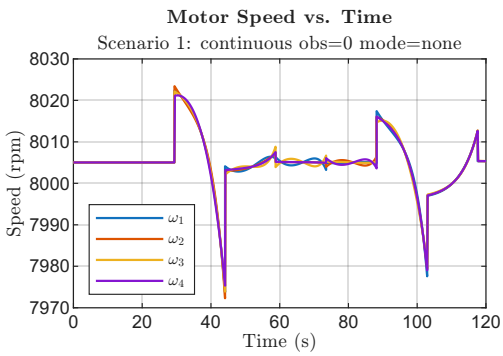

Fig. 8. Scenario 1 Motor Speeds vs. Time

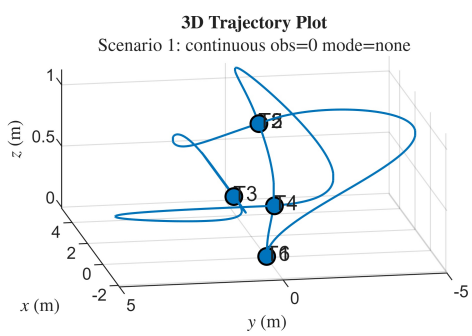

Fig. 9. Scenario 1 3D Trajectory

Scenario 2: Discrete-Time Receding Horizon Planning: Figs. 10, 11, and 12 show the results for the discrete-time receding horizon quadrotor.

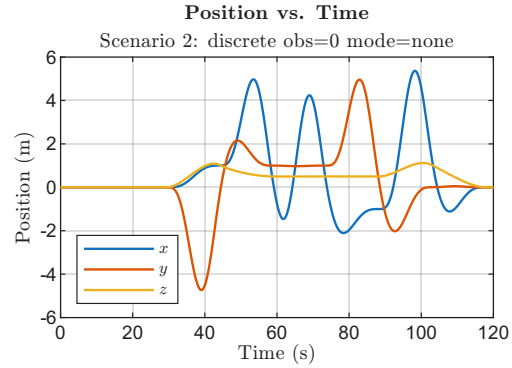

Fig. 10. Scenario 2 Position vs. Time

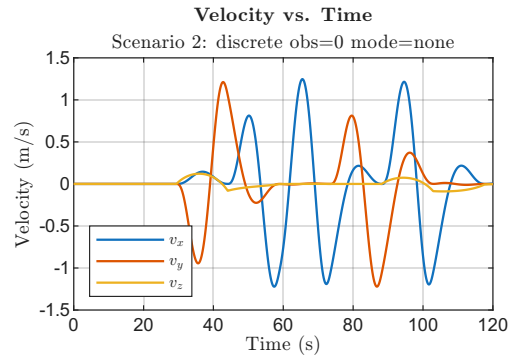

Fig. 11. Scenario 2 Velocity vs. Time

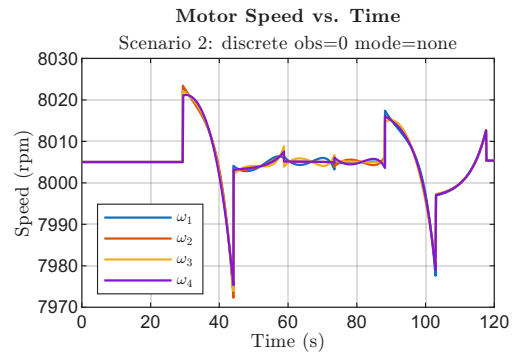

Fig. 12. Scenario 2 Motor Speeds vs. Time

Differences between Continuous and Discrete Control Results For discrete control, the Gramian is approximated by summing over a finite number of steps, which introduces numerical errors compared to the continuous case that computes the Gramian using the `finiteGramian` function.

When the step size was decreased too much, it increased the error. This could be explained by numerical issues such

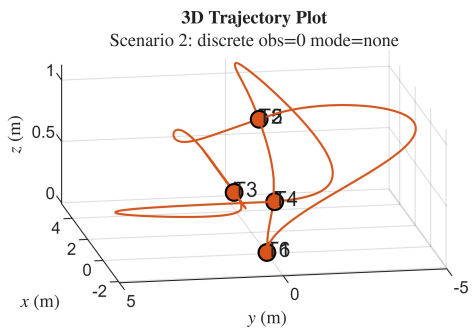

Fig. 13. Scenario 2 3D Trajectory

as the accumulation of errors and the poor resolution of system dynamics.

Reducing the regularization improved the accuracy of the discrete control results. From the results obtained, it was observed that the discrete observer exhibited a lower speed error but mostly had higher position errors.

Scenario 3: Continuous-Time Receding Horizon Planning with Luenberger Observer, Aggressive Mode: Figs. 14, 15, and 16 show the results for the continuous observer in the aggressive mode scenario.

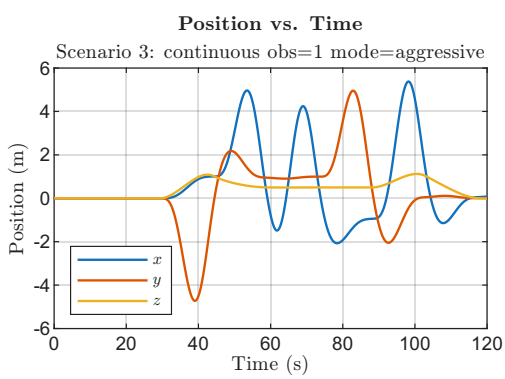

Fig. 14. Scenario 3 Position vs. Time

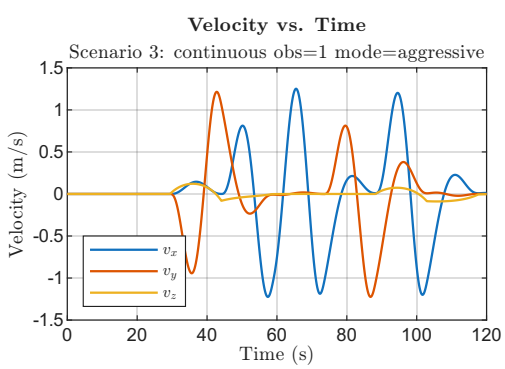

Fig. 15. Scenario 3 Velocity vs. Time

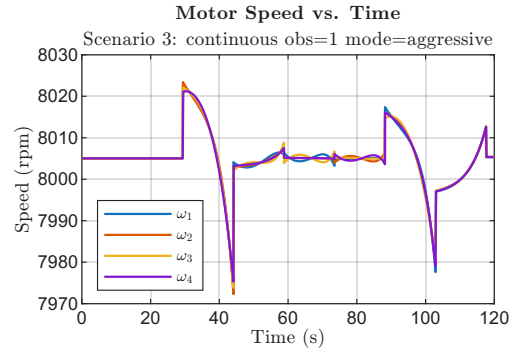

Fig. 16. Scenario 3 Motor Speeds vs. Time

Scenario 4: Continuous-Time Receding Horizon Planning with Luenberger, Slow Mode: Figs. 14, 15, and 16 show the results for the continuous observer in the slow mode scenario.

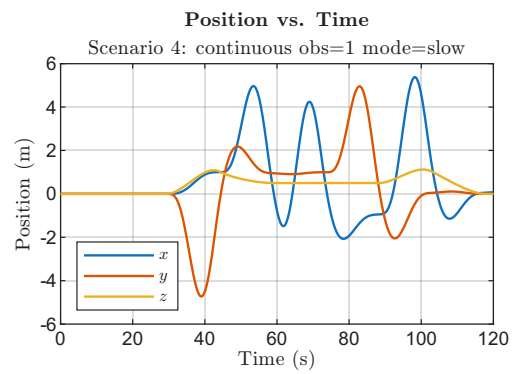

Fig. 17. Scenario 4 Position vs. Time

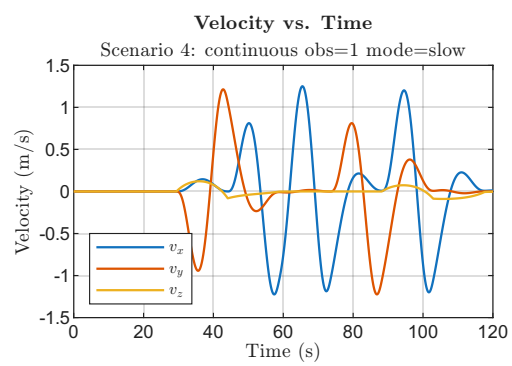

Fig. 18. Scenario 4 Velocity vs. Time

Differences between Continuous Time Aggressive Observer and Slow Observer The aggressive observer is designed to closely track the perfect observer, as its eigenvalues are placed to make the estimation error decay quickly. As seen Fig. 30, the RMS tracking error of the aggressive observer (higher observer bandwidth) was approximately equal to that of the perfect observer. Interestingly, the slower observer exhibited a slightly lower

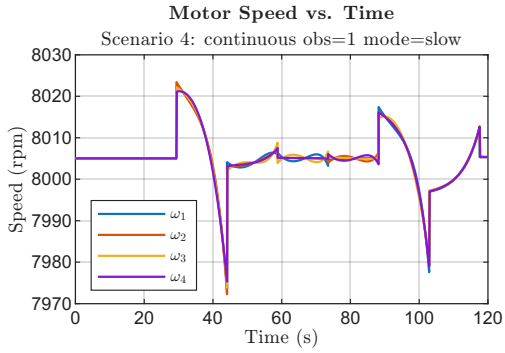

Fig. 19. Scenario 4 Motor Speeds vs. Time

RMS error than the aggressive observer, which could be explained by its more gradual response, possibly causing less overshoot.

Recommendations for horizon length, replan cadence, and observer pole placement for this vehicle

- Horizon Length - A greater horizon length was chosen to provide more accuracy in predictions.
- Replan Cadence - By trial and error, it was seen that a lower replan interval yielded less accurate position and velocity tracking. The replan interval was chosen to be closer to the horizon length, allowing for both position and velocity tracking.
- Observer Pole Placement - The results showed little difference in RMS errors between the aggressive and slow observers. The aggressive observer exhibited slightly higher RMS position errors, suggesting that selecting observer poles between the aggressive and slow configurations could improve performance.

Scenario 5: Discrete-Time Receding Horizon Planning with Luenberger Observer, Aggressive Mode: Figs. 20, 21, and 22 show the results for the discrete observer in the aggressive mode scenario.

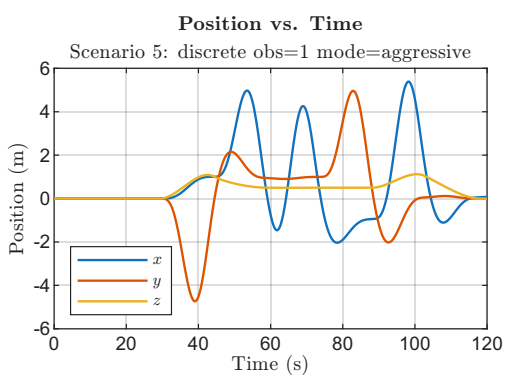

Fig. 20. Scenario 5 Position vs. Time

Scenario 6: Discrete-Time Receding Horizon Planning with Luenberger Observer, Slow Mode: Figs. 23, 24, and 25

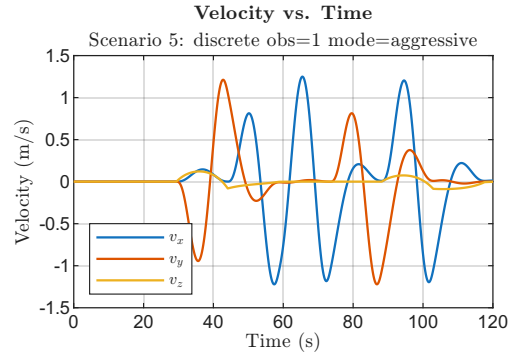

Fig. 21. Scenario 5 Velocity vs. Time

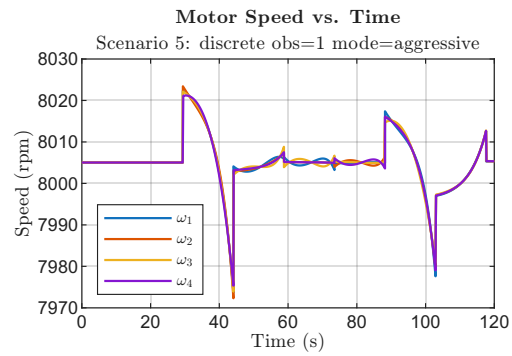

Fig. 22. Scenario 5 Motor Speeds vs. Time

show the results for the discrete observer in the slow mode scenario.

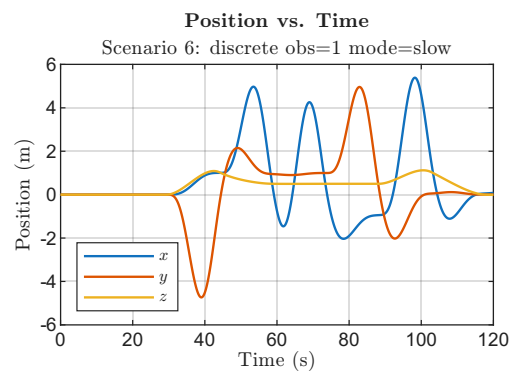

Fig. 23. Scenario 6 Position vs. Time

Differences between Discrete Time Aggressive Observer and Slow Observer Similar to continuous time, the aggressive observer is designed to closely track the perfect observer by making the estimation error decay quickly. The results showed varying behavior between the perfect discrete observer and the aggressive and slow observers, performing better at some instances and worse at others. There was also no significant difference between the slow and aggressive observers.

Recommendations Since there was no significant dif-

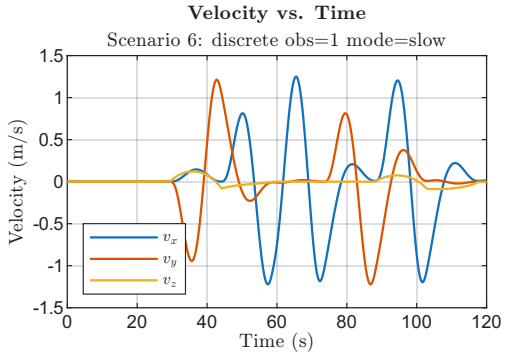

Fig. 24. Scenario 6 Velocity vs. Time

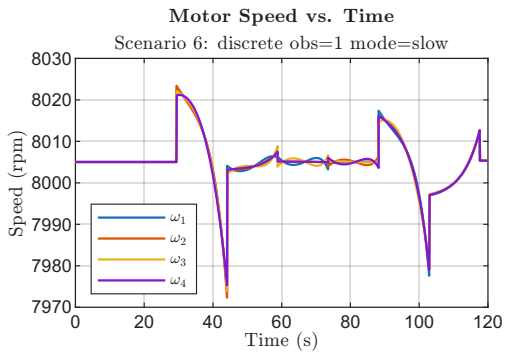

Fig. 25. Scenario 6 Motor Speeds vs. Time

ference between the aggressive and slow observers, pole placement can be made within the range of either observer.

Finally, Fig. 30 shows the estimated state error vs. time for all observer scenarios.

Realistic C Matrix

Replace $C = I$ with a realistic C_{struct} and run the observer with partial measurements.

Scenario 3, the continuous observer in aggressive mode was chosen to simulate the realistic C matrix. Results can be seen in Figs. 26, 27, 28, and 29.

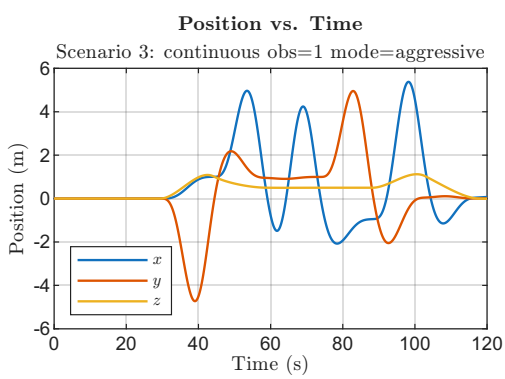

Fig. 26. Position vs. Time, Realistic C Matrix

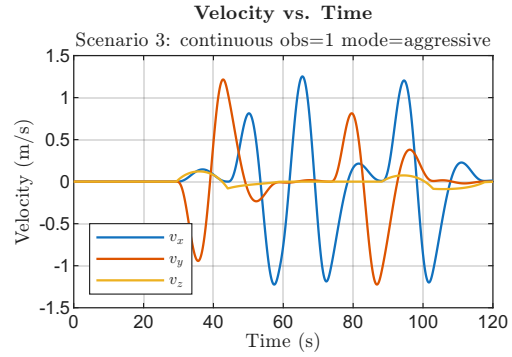

Fig. 27. Velocity vs. Time, Realistic C Matrix

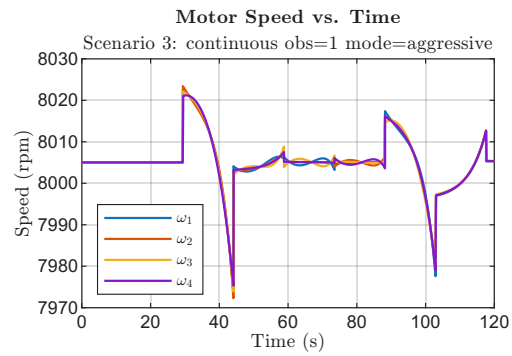

Fig. 28. Motor Speeds vs. Time, Realistic C Matrix

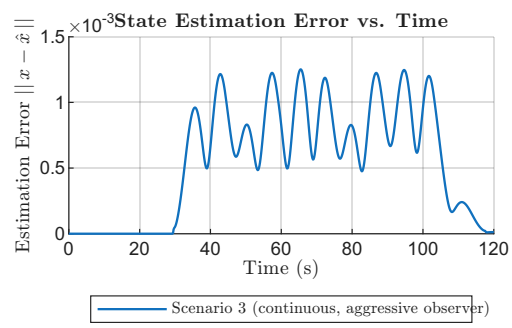

Fig. 29. State Estimation Error vs. Time, Realistic C Matrix

Difference between $C = I$ vs realistic C_{struc} With the updated C matrix, the quadrotor successfully met all position and velocity waypoints.

Comparing the results with the previous C matrix, there were only slight differences.This could be explained by the observability matrix having full rank in both cases.

Recommendations The previous horizon length, replan cadence, and waypoint hold achieved the desired waypoints with the new C matrix. Similar to the previous part, due to the similar trends observed, tuning the observer poles between the aggressive and slow configurations could improve performance.

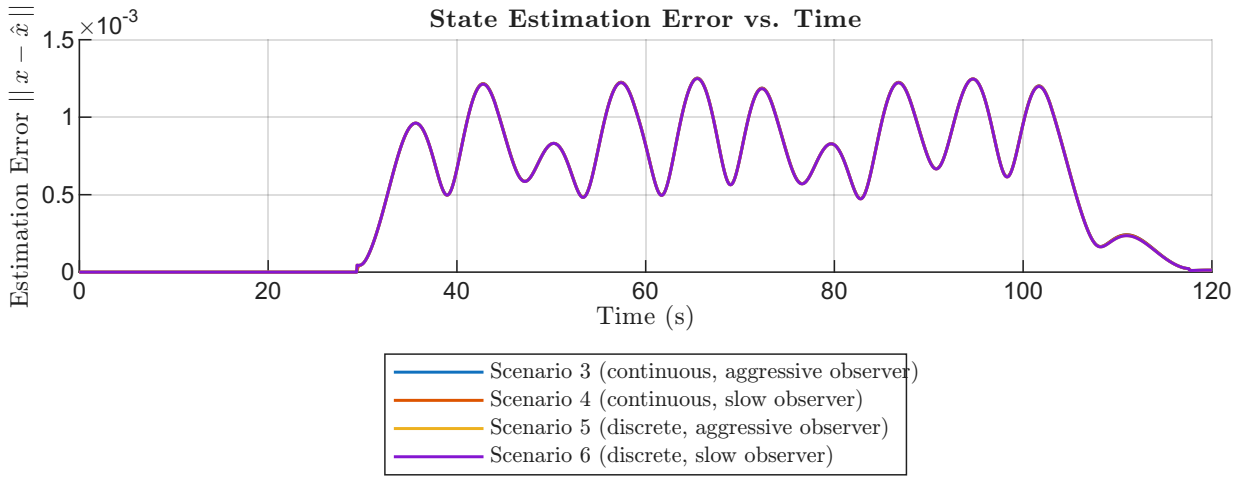

Fig. 30. State Estimation Error for Observer Scenarios

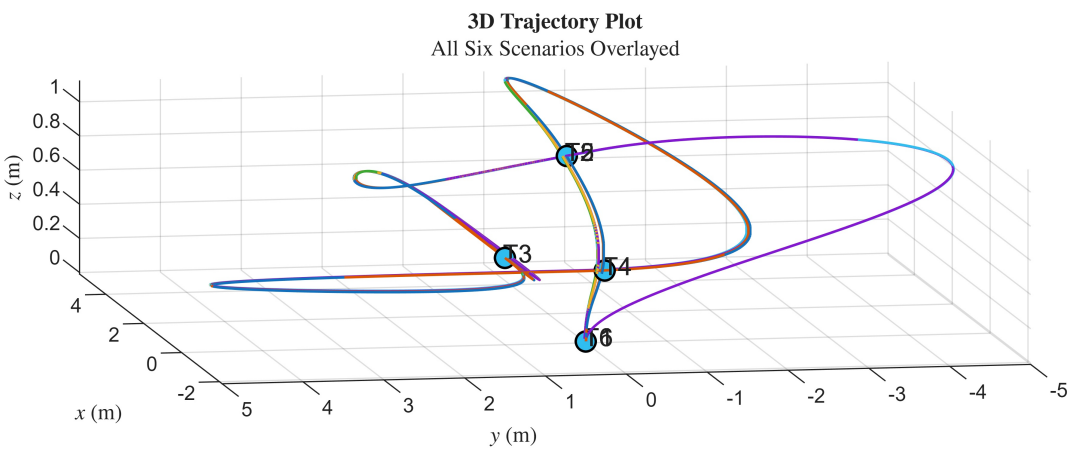

Fig. 31. 3D Trajectory Plot, All Scenarios

TABLE I: Position and Velocity Tracking Metrics for Scenarios 1–6

Scenario 1 – Position Metrics						
Target Pos	Reached Pos	minDist	Reached	RMS Pos	Overshoot	t_{settle}
[0;0;0]	[0;0;0]	0	1	0	5.4837	0.00
[1;0;1]	[1.0023;5.7E−5;1.0016]	0.0028	1	2.0531	5.5315	44.323
[0;1;0.5]	[1.9E−4;1.0005;0.5]	5.7E−4	1	2.8444	5.5892	14.597
[-1;0;0.5]	[-0.9990;-3.1E−4;0.5014]	0.0017	1	3.2041	6.3995	14.593
[1;0;1]	[1;0.0042;0.9998]	0.0042	1	2.8816	2.1855	14.606
[0;0;0]	[0.0011;-7.4E−4;1E−5]	0.0014	1	0.8415	0.0098	11.603
Scenario 1 – Velocity Metrics						
Target Vel	Reached Vel	speedErr	angDeg	dirOK	magOK	Status
[0;0;0]	[0;0;0]	0	—	1	1	OK (hover)
[0;1;-0.1]	[8.5E−4;1.0019;-0.078]	8.5E−5	5.84	1	1	angle&speed OK
[-1;0;0]	[-1; -9.2E−5; -0.0017]	6.1E−5	0.099	1	1	angle&speed OK
[0;-1;0]	[6.6E−4; -1; 0.0074]	8.4E−5	0.423	1	1	angle&speed OK
[-1;0;-0.1]	[-1.0012; 5E−4; -0.086]	5.4E−5	5.76	1	1	angle&speed OK
[0;0;0]	[0.002; -3.7E−4;-0.004]	0.0045	—	1	1	OK (hover)
Scenario 2 – Position Metrics						

Continued on next page

Target Pos	Reached Pos	minDist	Reached	RMS Pos	Overshoot	t_{settle}
[0;0;0]	[0;0;0]	0	1	0	5.4908	0.00
[1;0;1]	[1.0045;3.7E−4;1]	0.0045	1	2.0553	5.5302	44.344
[0;1;0.5]	[-4.7E−4;0.99997;0.49965]	5.9E−4	1	3.2270	5.5944	14.593
[-1;0;0.5]	[-1.0018;1.2E−4;0.50153]	0.0024	1	3.2204	6.4062	29.299
[1;0;1]	[1.0005;0.00518;0.99767]	0.0057	1	2.8783	2.1491	14.611
[0;0;0]	[-0.00134;-0.00190;2E−4]	0.0023	1	0.82588	0.0076	11.574

Scenario 2 – Velocity Metrics

Target Vel	Reached Vel	speedErr	angDeg	dirOK	magOK	Status
[0;0;0]	[0;0;0]	0	—	1	1	OK (hover)
[0;1;-0.1]	[0.0011;1.0019;-0.0783]	7E−5	5.843	1	1	angle&speed OK
[-1;0;0]	[-1;1.7E−4;-0.0017]	2.7E−5	0.100	1	1	angle&speed OK
[0;-1;0]	[1.1E−4;-1;0.00698]	2.1E−5	0.400	1	1	angle&speed OK
[-1;0;-0.1]	[-1.0011;6.4E−4;-0.086]	2.1E−4	5.765	1	1	angle&speed OK
[0;0;0]	[0.00186;-0.00059;-0.00382]	0.00429	—	1	1	OK (hover)

Scenario 3 – Position Metrics

Target Pos	Reached Pos	minDist	Reached	RMS Pos	Overshoot	t_{settle}
[0;0;0]	[0;0;0]	0	1	0	5.5023	0.00
[1;0;1]	[1.0023;5.7E−5;1.0016]	0.0028	1	2.0531	5.5124	44.323
[0;1;0.5]	[-3.2E−4;0.99641;0.49999]	0.0036	1	2.8480	5.6025	14.614
[-1;0;0.5]	[-0.94097;1.9E−4;0.50136]	0.0590	1	3.1991	6.3614	14.562
[1;0;1]	[1.0005;0.05250;0.9995]	0.0525	1	2.8840	2.1340	14.624
[0;0;0]	[0.03641;0.00573;0.02682]	0.0456	1	0.8870	0.0350	11.421

Scenario 3 – Velocity Metrics

Target Vel	Reached Vel	speedErr	angDeg	dirOK	magOK	Status
[0;0;0]	[0;0;0]	0	—	1	1	OK (hover)
[0;1;-0.1]	[8.48E−4;1.0019;-0.0783]	8.57E−5	5.844	1	1	angle&speed OK
[-1;0;0]	[-0.99995;-0.00810;-0.00172]	1.1E−5	0.474	1	1	angle&speed OK
[0;-1;0]	[0.0098;-0.99998;0.00736]	5.27E−5	0.702	1	1	angle&speed OK
[-1;0;-0.1]	[-1.0013;0.00809;-0.08629]	5.77E−5	5.783	1	1	angle&speed OK
[0;0;0]	[0.00880;9.34E−4;-0.00557]	0.0105	—	1	1	OK (hover)

Scenario 4 – Position Metrics

Target Pos	Reached Pos	minDist	Reached	RMS Pos	Overshoot	t_{settle}
[0;0;0]	[0;0;0]	0	1	0	5.5017	0.00
[1;0;1]	[1.0023;5.7E−5;1.0016]	0.0028	1	2.0531	5.5130	44.323
[0;1;0.5]	[3.8E−5;0.99659;0.49999]	0.0034	1	2.8479	5.6022	14.613
[-1;0;0.5]	[-0.94293;1.5E−4;0.50136]	0.0571	1	3.1993	6.3627	14.563
[1;0;1]	[1.0004;0.05069;0.9995]	0.0507	1	2.8839	2.1359	14.622
[0;0;0]	[0.03587;0.00542;0.02682]	0.0444	1	0.88562	0.03369	11.425

Scenario 4 – Velocity Metrics

Target Vel	Reached Vel	speedErr	angDeg	dirOK	magOK	Status
[0;0;0]	[0;0;0]	0	—	1	1	OK (hover)
[0;1;-0.1]	[8.48E−4;1.0019;-0.0782]	8.57E−5	5.844	1	1	angle&speed OK
[-1;0;0]	[-0.99997;-0.00780;-0.00172]	2.45E−6	0.458	1	1	angle&speed OK
[0;-1;0]	[0.00950;-0.99998;0.00736]	5.55E−5	0.689	1	1	angle&speed OK
[-1;0;-0.1]	[-1.0013;0.00781;-0.08629]	7.37E−5	5.781	1	1	angle&speed OK
[0;0;0]	[0.00853;8.85E−4;-0.00555]	0.01021	—	1	1	OK (hover)

Scenario 5 – Position Metrics

Target Pos	Reached Pos	minDist	Reached	RMS Pos	Overshoot	t_{settle}
[0;0;0]	[0;0;0]	0	1	0	5.5104	0.00
[1;0;1]	[1.0045;3.7E−4;1]	0.0045	1	2.0553	5.5103	44.344
[0;1;0.5]	[3.6E−4;0.99724;0.49999]	0.0028	1	2.8529	5.6093	14.612
[-1;0;0.5]	[-0.94007;6.5E−4;0.50153]	0.0600	1	3.2013	6.3688	14.556
[1;0;1]	[1.0009;0.05707;0.99741]	0.0571	1	2.8809	2.0949	14.632

Continued on next page

[0;0;0]	[0.03736;0.004996;0.02721]	0.0465	1	0.87032	0.03540	11.380

Scenario 5 – Velocity Metrics

Target Vel	Reached Vel	speedErr	angDeg	dirOK	magOK	Status
[0;0;0]	[0;0;0]	0	—	1	1	OK (hover)
[0;1;-0.1]	[0.0011;1.0019;-0.0783]	7.04E−5	5.843	1	1	angle&speed OK
[-1;0;0]	[-1;-0.00838;-0.00174]	4.49E−5	0.490	1	1	angle&speed OK
[0;-1;0]	[0.00978;-0.99991;0.00698]	1.42E−5	0.689	1	1	angle&speed OK
[-1;0;-0.1]	[-0.99613;0.00877;-0.08623]	0.00509	5.783	1	1	angle&speed OK
[0;0;0]	[0.00898;7.74E−4;-0.00560]	0.01062	—	1	1	OK (hover)

Scenario 6 – Position Metrics

Target Pos	Reached Pos	minDist	Reached	RMS Pos	Overshoot	t_{settle}
[0;0;0]	[0;0;0]	0	1	0	5.5103	0.00
[1;0;1]	[1.0045;3.7E−4;1]	0.0045	1	2.0553	5.5103	44.344
[0;1;0.5]	[2.3E−4;0.99725;0.49999]	0.0028	1	2.8529	5.6093	14.612
[-1;0;0.5]	[-0.94023;6.5E−4;0.50153]	0.0598	1	3.2013	6.3689	14.556
[1;0;1]	[1.0008;0.05694;0.99741]	0.0570	1	2.8808	2.0951	14.632
[0;0;0]	[0.03731;0.00497;0.02711]	0.0464	1	0.87022	0.03529	11.381

Scenario 6 – Velocity Metrics

Target Vel	Reached Vel	speedErr	angDeg	dirOK	magOK	Status
[0;0;0]	[0;0;0]	0	—	1	1	OK (hover)
[0;1;-0.1]	[0.0011;1.0019;-0.0783]	7.04E−5	5.843	1	1	angle&speed OK
[-1;0;0]	[-1;-0.00836;-0.00174]	4.60E−5	0.489	1	1	angle&speed OK
[0;-1;0]	[0.00976;-0.99991;0.00698]	1.40E−5	0.687	1	1	angle&speed OK
[-1;0;-0.1]	[-0.99613;0.00875;-0.08623]	0.00509	5.783	1	1	angle&speed OK
[0;0;0]	[0.00896;7.71E−4;-0.00560]	0.01060	—	1	1	OK (hover)

Realistic C Matrix – Position Metrics (Continuous Observer, Aggressive)

Target Pos	Reached Pos	minDist	Reached	RMS Pos	Overshoot	t_{settle}
[0; 0; 0]	[0; 0; 0]	0	1	0	5.5011	0.00
[1; 0; 1]	[1.0023; 5.712E-5; 1.0016]	0.0028	1	2.0531	5.5136	44.323
[0; 1; 0.5]	[4.4197E-4; 0.99664; 0.49999]	0.00339	1	2.8477	5.6016	14.612
[-1; 0; 0.5]	[-0.94483; 1.4434E-4; 0.50136]	0.05519	1	3.1994	6.3640	14.564
[1; 0; 1]	[1.0003; 0.050181; 0.9995]	0.05018	1	2.8838	2.1366	14.622
[0; 0; 0]	[0.035336; 0.0052938; 0.024427]	0.04328	1	0.88435	0.03248	11.430

Realistic C Matrix – Velocity Metrics (Continuous Observer, Aggressive)

Target Vel	Reached Vel	speedErr	angDeg	dirOK	magOK	Status
[0; 0; 0]	[0; 0; 0]	0	—	1	1	OK (hover speed)
[0; 1; -0.1]	[8.4812E-4; 1.0019; -0.078249]	8.567E−5	5.8444	1	1	angle & speed OK
[-1; 0; 0]	[-0.99999; -0.0077216; -0.0017226]	1.741E−5	0.45329	1	1	angle & speed OK
[0; -1; 0]	[0.0092099; -0.99998; 0.0073606]	5.428E−5	0.67549	1	1	angle & speed OK
[-1; 0; -0.1]	[-1.0013; 0.0077352; -0.08629]	9.086E−5	5.7809	1	1	angle & speed OK
[0; 0; 0]	[0.008263; 0.00087137; -0.0055288]	0.00998	—	1	1	OK (hover speed)

References

[1] Brian Douglas, "What Is LQR Optimal Control? | State Space, Part 4." MathWorks, March 2019.

Appendix

Attachment: Full MATLAB Script

Part 1

```
clear; clc;
%% System Model (Shared for all suites)
g = 9.81;
m = 1.0;
Ixx = 0.02;
Iyy = 0.02;
Izz = 0.04;
A = zeros(12);
A(1,7)=1;  A(2,8)=1;  A(3,9)=1;
A(4,10)=1; A(5,11)=1; A(6,12)=1;
A(7,5)=g;
A(8,4)=-g;
drag_trans = 0.2;
drag_rot = 0.05;
A(7,7)  = -drag_trans;
A(8,8)  = -drag_trans;
A(9,9)  = -drag_trans;
A(10,10) = -drag_rot;
A(11,11) = -drag_rot;
A(12,12) = -drag_rot;
t = linspace(0,5,500);
%% ======================= Suite 1
    =======================
C1 = zeros(6,12);
C1(1,1)=1;
C1(2,2)=1;
C1(3,3)=1;
C1(4,4)=1;
C1(5,5)=1;
C1(6,6)=1;
C = C1;
Ob = obsv(A, C);
r = rank(Ob);
fprintf('Suite 1: Rank of observability matrix =
    %d (out of 12)\n', r);
```

```
Suite 1: Rank of observability matrix
         = 12 (out of 12)
```

```
W = zeros(12);
for k = 1:length(t)
   dt = t(2) - t(1);
   W = W + expm(A'*t(k)) * (C'*C) * expm(A*t(k))
       * dt;
end
e = eig(W);
figure;
    semilogy(e,'o','LineWidth',3);
title('\textbf{Suite 1: Eigenvalues of
    Observability Gramian}','Interpreter','latex'
    );
xlabel('Index','Interpreter','latex');
ylabel('Eigenvalue (log scale)','Interpreter','
    latex');
grid on;
formatFigure('single')
```

```
%% ======================= Suite 2
    =======================
C2 = zeros(7,12);
C2(1,3)=1;
C2(2:4,4:6)=eye(3);
C2(5:7,10:12)=eye(3);
C = C2;
Ob = obsv(A, C);
r = rank(Ob);
fprintf('Suite 2: Rank of observability matrix =
    %d (out of 12)\n', r);
```

```
Suite 2: Rank of observability matrix
         = 8 (out of 12)
```

```
W = zeros(12);
for k = 1:length(t)
   dt = t(2) - t(1);
   W = W + expm(A'*t(k)) * (C'*C) * expm(A*t(k))
       * dt;
end
e = eig(W);
figure;
semilogy(e,'o','LineWidth',3);
title('\textbf{Suite 2: Eigenvalues of
    Observability Gramian}','Interpreter','latex'
    );
xlabel('Index','Interpreter','latex');
ylabel('Eigenvalue (log scale)','Interpreter','
    latex');
grid on;
formatFigure('single')
```

```
%% ======================= Suite 3
    =======================
C3 = zeros(8,12);
% GPS: x, y, z
C3(1,1) = 1;   % x
C3(2,2) = 1;   % y
C3(3,3) = 1;   % z
% Magnetometer: psi
C3(4,6) = 1;   % psi
% Optical flow: x_dot, y_dot
C3(5,7) = 1;   % x_dot
C3(6,8) = 1;   % y_dot
% Gyros: p, q
C3(7,10) = 1;  % p
C3(8,11) = 1;  % q
C = C3;
```

```
Ob = obsv(A, C);
r = rank(Ob);
fprintf('Suite 3: Rank of observability matrix =
    %d (out of 12)\n', r);
```

```
Suite 3: Rank of observability matrix
         = 12 (out of 12)
```

```
W = zeros(12);
for k = 1:length(t)
   dt = t(2) - t(1);
   W = W + expm(A'*t(k)) * (C'*C) * expm(A*t(k))
       * dt;
end

% similar loop to compute discrete Gramian
N  = 300;
Wd = zeros(12);
Ad = eye(12) + A*0.01;
% Wd = dlyap(Ad', C'*C);
AdT = Ad';
Qd  = C3'*C3;
for k = 0:N-1
    Ak  = Ad^k;
    Wd  = Wd + (AdT^k) * Qd * Ak;
end
```

```
e = eig(W);
figure;
semilogy(e,'o','LineWidth',3);

grid on; hold on;

% Ad =0.999*(eye(12) + A*0.01); Bd = zeros(12,1);
% Wd = dlyap(Ad', C3'*C3);
e1 = eig(Wd);
semilogy(e1,'o','LineWidth',3);
title('\textbf{Suite 3: Eigenvalues of
    Observability Gramian}','Interpreter','latex'
    );
xlabel('Index','Interpreter','latex');
ylabel('Eigenvalue (log scale)','Interpreter','
    latex');
legend('Continuous','Discrete','Interpreter','
    latex','location','best');
formatFigure('single')
```

Part 2

```
clc;clear;close all;
p0 = [0, 0, 0];     v0 = [0, 0, 0];        %
    starting point p0 and velocities v0
p1 = [1, 0, 1];     v1 = [0, 1, -0.1];     %
    waypoint 1 and velocities at waypoint 1
p2 = [0, 1, 0.5];   v2 = [-1, 0, 0];       %
    waypoint 2 and velocities at waypoint 2
p3 = [-1, 0, 0.5]; v3 = [0, -1, 0];        %
    waypoint 3 and velocities at waypoint 3
p4 = [1, 0, 1];     v4 = [-1, 0, -0.1];    %
    waypoint 4 and velocities at waypoint 4
p5 = [0, 0, 0];     v5 = [0, 0, 0];        % final
    stop waypoint 5 and velocities at waypoint 5

%% ---------------- User / vehicle parameters
    ----------------
g = 9.81; m = 1.0;
L = 0.25;
Ixx = 0.02; Iyy = 0.02; Izz = 0.04;

% Linearized A,B (12x12, 12x4)
n = 12; A = zeros(n);
A(1,7)=1; A(2,8)=1; A(3,9)=1;
A(4,10)=1; A(5,11)=1; A(6,12)=1;
A(7,5)= g; A(8,4)=-g; % see linearized hover
    model above, models (x,y) acceleration
drag_trans = 0.2; drag_rot = 0.05;
A(7,7) = -drag_trans; A(8,8) = -drag_trans; A
    (9,9) = -drag_trans;
A(10,10) = -drag_rot; A(11,11)=-drag_rot; A
    (12,12)=-drag_rot;
% Parameters related to the physics model of
    propeller thrust from motors
kT = 3.49e-6; kQ = 2.0e-8;     % (not used
    directly for linear B scaling)
omega0 = sqrt(m*g/(4*kT));
omega0_vec = omega0 * ones(4,1);
dT_domega = 2*kT*omega0; dTau_domega = 2*kQ*
    omega0;
% motor limits rad/sec max speed = 12000 rpm
omega_min = 0; omega_max = 12000 * 2*pi/60;
% B mapping (linearized: d_omega -> accel)
B = zeros(n,4);
B(9,:)  = (dT_domega/m) * [1 1 1 1];             %
     z_ddot
B(10,:) = (dT_domega * L / Ixx) * [0 1 0 -1];   %
    p_dot
B(11,:) = (dT_domega * L / Iyy) * [1 0 -1 0];   %
    q_dot
B(12,:) = (dTau_domega / Izz) * [1 -1 1 -1];    %
    r_dot

% C3 matrix from Part 1
C3 = zeros(8,12);
% GPS: x, y, z
C3(1,1) = 1;   % x
C3(2,2) = 1;   % y
C3(3,3) = 1;   % z
% Magnetometer: psi
C3(4,6) = 1;   % psi
% Optical flow: x_dot, y_dot
C3(5,7) = 1;   % x_dot
C3(6,8) = 1;   % y_dot
% Gyros: p, q
C3(7,10) = 1;  % p
C3(8,11) = 1;  % q

% LQR feedback for robustness around feedforward
    (tune)
Qk = diag([0.5 9 17 1 1 1 1 1 1 1 1 1]);
Rk = 0.6*eye(4);
K_lqr = 0.1*lqr(A,B,Qk,Rk);

%% ---------------- Trajectory waypoints &
    targets ----------------
% Each target = [pos(3); vel(3); angles(3)=0; ang
     rates(3)=0]  (12x1)
makeTarget = @(pos, vel) [pos(:); zeros(3,1); vel
    (:); zeros(3,1)];

targets{1} = makeTarget(p0, v0);
targets{2} = makeTarget(p1, v1);
targets{3} = makeTarget(p2, v2);
targets{4} = makeTarget(p3, v3);
targets{5} = makeTarget(p4, v4);
targets{6} = makeTarget(p5, v5);


% waypoint durations to attempt (s) - students
    can vary
waypoint_hold = 20;   % seconds before forcing
    move to next (upper bound)

%% ---------------- Simulation & receding-horizon
     params ----------------
dt = 1e-3;                      % integration step
Tsim_total = 6*waypoint_hold;  % simulate through
     all waypoints
tvec = 0:dt:Tsim_total;
Nsim = numel(tvec);


Tf = 15;                        % planning horizon
    (s)
Nf = round(Tf/dt);
replan_interval = 14.7;          % seconds
replan_steps = round(replan_interval/dt);
eps_reg = 1e-10;                 % Gramian
    regularization

%% ---------------- Observer pole sets (
    continuous & discrete) ----------
% continuous observer poles (for place())
obs_poles_aggressive = -[8:19];   % 12 poles
obs_poles_slow = -[2:13];

% For discrete case, choose poles inside unit
    circle (e.g., 0.2..0.7)
disc_poles_aggr = 0.15 * ones(1,n);   % fast
    discrete
disc_poles_slow = 0.7 * ones(1,n);    % slow
    discrete

%% ---------------- Prep logging containers
```

```
    ----------------
results = struct();

scenarios = { ...
    struct('type','continuous','useLQR',false,'
        useObserver',false,'obsMode','none'), ...
    struct('type','discrete'  ,'useLQR',false,'
        useObserver',false,'obsMode','none'), ...
    struct('type','continuous','useLQR',false,'
        useObserver',true ,'obsMode','aggressive'
        ), ...
    struct('type','continuous','useLQR',false,'
        useObserver',true ,'obsMode','slow'), ...
    struct('type','discrete'  ,'useLQR',false,'
        useObserver',true ,'obsMode','aggressive'
        ), ...
    struct('type','discrete'  ,'useLQR',false,'
        useObserver',true ,'obsMode','slow') ...
};

% add scenario index field - easier to label
    plots later
for k = 1:numel(scenarios)
    scenarios{k}.id = k;
end

% you can reduce scenarios for quicker runs

scenarios = scenarios(3:6);

%% -------------- Helper: finite-horizon Gramian
    (continuous) -----------
function [W,info] = finiteGramian(A,B,T,dt)
    N = max(2, round(T/dt));
    W = zeros(size(A));
    BBt = B*B';
    for k = 1:N
        s = (k-1)*(T/(N-1));
        Phi = expm(A*s);
        if k==1 || k==N
            w = 0.5;
        else
            w = 1.0;
        end
        W = W + w * (Phi * BBt * Phi') * (T/(N-1)
            );
    end
    info.dt = T/(N-1); info.cond = cond(W); info.
        rank = rank(W);
end

%% -------------- Helper: discrete Bd approx
    ----------------------------
function Bd = computeBd(A,B,dt)
    % use numerical quadrature for integral \
        int_0^dt exp(A s) B ds
    m = 10;
    S = zeros(size(B));
    for j=0:(m-1)
        s = (j+0.5)/m * dt;
        S = S + expm(A*s) * B;
    end
    Bd = S * (dt/m);
end

%% ---------------- Main loop over scenarios
    ----------------
for si = 1:numel(scenarios)
    sc = scenarios{si};
    fprintf('\nRunning scenario %d/%d: Scenario %
        d: %s, useLQR=%d, useObserver=%d, mode=%s
        \n', si, numel(scenarios), sc.id, sc.type
        , sc.useLQR, sc.useObserver, sc.obsMode);

    % initialize simulation variables
    x = zeros(n,1);   % start at origin rest
    xhat = zeros(n,1);
    omega_cmd = omega0_vec;

    Xlog = zeros(n,Nsim);
    Xhatlog = zeros(n,Nsim);
    OmegaLog = zeros(4,Nsim);
    targetIdx = 1;
    t_since_switch = 0;

    % discrete-time precompute if needed
    if strcmp(sc.type,'discrete')
        Ad = expm(A*dt);
        Bd = computeBd(A,B,dt);
    end

    % observer gain
    if sc.useObserver
        if strcmp(sc.type,'continuous')
            if strcmp(sc.obsMode,'aggressive')
                % L = place(A', C3',
                    obs_poles_aggressive)' ;
                    % C3 matrix in Part 1
                L = place(A', eye(n)',
                    obs_poles_aggressive)' ;
            else
                L = place(A', eye(n)',
                    obs_poles_slow)' ;
            end
        else
            % discrete observer via place on Ad,
                Cd
            Cd = eye(n);
            if strcmp(sc.obsMode,'aggressive')
                Ld = place(Ad', Cd',
                    disc_poles_aggr)';   %
                    discrete
            else
                Ld = place(Ad', Cd',
                    disc_poles_slow)';
            end
        end
    end

    % receding horizon vars
    next_replan = 1;
    domega_ff = zeros(4,Nf);
    ff_step = 1;

    for k = 1:Nsim
        tk = tvec(k);
        target = targets{targetIdx};

        % check if we should switch to next
            waypoint (simple timed switch)
        if t_since_switch >= waypoint_hold
            targetIdx = min(targetIdx+1, numel(
                targets));
            t_since_switch = 0;
            fprintf('Switch to target %d at t=%.2
                f\n', targetIdx, tk);
        end

        % Replan condition -- current horizon
            point has been reached
        if k >= next_replan
            % compute Gramian & feedforward for
                current target using current
                state or estimate
            if strcmp(sc.type,'continuous')
                [W,info] = finiteGramian(A,B,Tf,
                    dt);
                W = W + eps_reg * eye(n);
```

```
            PhiT = expm(A*Tf);
            if sc.useObserver
                x0for = xhat;
            else
                x0for = x;
            end
            b = target - PhiT * x0for;
            for kk = 1:Nf
                t_rel = (kk-1)*dt;
                domega_ff(:,kk) = real(B' *
                    expm(A'*(Tf - t_rel)) * (
                    W \ b));
            end
        else
            % discrete-time Gramian
            % compute Wd = sum_{i=0}^{Nf-1}
                Ad^i Bd Bd' (Ad^i)'
            Wd = zeros(n);
            Ak = eye(n);
            for ii = 0:Nf-1
                Wd = Wd + Ak * (Bd * Bd') *
                    Ak';
                Ak = Ad * Ak;
            end
            Wd = Wd + eps_reg*eye(n);
            AdN = Ad^Nf;
            if sc.useObserver
                x0for = xhat;
            else
                x0for = x;
            end
            b = target - AdN * x0for;
            % compute feedforward u_k (time-
                varying) using discrete
                formula:
            for kk = 1:Nf
                krel = kk-1;
                E = (Ad')^(Nf - krel);
                domega_ff(:,kk) = real(Bd' *
                    E * (Wd \ b));
            end
        end

        next_replan = k + replan_steps;
        ff_step = 1;
    end

    % feedforward & feedback
    if ff_step <= Nf
        ff_now = domega_ff(:,ff_step);
    else
        ff_now = zeros(4,1);
    end
    if sc.useLQR
         if sc.useObserver
            e = xhat - target;
        else
            e = x - target;
        end
        u_fb = -K_lqr * e;
    else
        % do not use LQR control feedback
        u_fb = zeros(4,1);
    end
    d_omega = ff_now + u_fb;
    omega_cmd = omega0_vec + d_omega;
    omega_cmd = min(max(omega_cmd, omega_min)
        , omega_max);

    % plant update
    if strcmp(sc.type,'continuous')
        xdot = A*x + B*(omega_cmd -
            omega0_vec);
        x = x + xdot * dt;
    else
        % discrete step
        x = Ad * x + Bd * (omega_cmd -
            omega0_vec);
    end

    % measurement (full-state perfect but add
         tiny noise optional)
    y = eye(n) * x;

    % measurement with C3, part 1
    % y = C3 * x;

    % observer update
    if sc.useObserver
        if strcmp(sc.type,'continuous')
            xhat = xhat + (A*xhat + B*(
                omega_cmd - omega0_vec) + L*(
                y ...
            - xhat)) * dt;
            % xhat = xhat + (A*xhat + B*(
                omega_cmd - omega0_vec) + L*(
                y ...
            % - C3*xhat)) * dt;   % C3, part
                1
        else
            % discrete observer: xhat[k+1] =
                Ad xhat + Bd u + Ld*(y - Ad*
                xhat - Bd*u?)
            % Use standard discrete observer:
                 xhat_{k+1} = Ad*xhat_k + Bd*
                u_k + Ld*(y_k - C*xhat_k)
            xhat = Ad * xhat + Bd * (
                omega_cmd - omega0_vec) + Ld
                *(y - xhat);
        end
    end

    % logs
    Xlog(:,k) = x; Xhatlog(:,k) = xhat;
        OmegaLog(:,k) = omega_cmd * 60/(2*pi)
        ;
    % progress
    ff_step = ff_step + 1;
    t_since_switch = t_since_switch + dt;
end

% store results
results(si).scenario = sc;
results(si).t = tvec;
results(si).X = Xlog;
results(si).Xhat = Xhatlog;
results(si).Omega = OmegaLog;
fprintf('Scenario %d done.\n', si);
end
```

```
Running scenario 1/4: Scenario 3: continuous,
useLQR=0, useObserver=1, mode=aggressive

Switch to target 2 at t=20.00
Switch to target 3 at t=40.00
Switch to target 4 at t=60.00
Switch to target 5 at t=80.00
Switch to target 6 at t=100.00
Switch to target 6 at t=120.00
Scenario 1 done.

Running scenario 2/4: Scenario 4: continuous,
useLQR=0, useObserver=1, mode=slow

Switch to target 2 at t=20.00
```

```
Switch to target 3 at t=40.00
Switch to target 4 at t=60.00
Switch to target 5 at t=80.00
Switch to target 6 at t=100.00
Switch to target 6 at t=120.00
Scenario 2 done.

Running scenario 3/4: Scenario 5: discrete,
useLQR=0, useObserver=1, mode=aggressive
Switch to target 2 at t=20.00
Switch to target 3 at t=40.00
Switch to target 4 at t=60.00
Switch to target 5 at t=80.00
Switch to target 6 at t=100.00
Switch to target 6 at t=120.00
Scenario 3 done.

Running scenario 4/4: Scenario 6: discrete,
useLQR=0, useObserver=1, mode=slow
Switch to target 2 at t=20.00
Switch to target 3 at t=40.00
Switch to target 4 at t=60.00
Switch to target 5 at t=80.00
Switch to target 6 at t=100.00
Switch to target 6 at t=120.00
Scenario 4 done.
```

2D Plots

```
%% Plot example results
numScenarios = numel(scenarios);

% Compute subplot grid size dynamically
nCols = ceil(sqrt(numScenarios));
nRows = ceil(numScenarios / nCols);

% show positions for each scenario
figure('Name','Position traces for scenarios','
    NumberTitle','off');
for si = 1:numel(scenarios)
    % subplot(nRows, nCols, si);
    plot(results(si).t, results(si).X(1,:),
        results(si).t, results(si).X(2,:),
        results(si).t, results(si).X(3,:),'
        LineWidth',1.5);
    grid on;
    title('\textbf{Position vs. Time}','
        Interpreter','latex');
    subtitle(sprintf('Scenario %d: %s obs=%d mode
        =%s', results(si).scenario.id, results(si
        ).scenario.type, results(si).scenario.
        useObserver, results(si).scenario.obsMode
        ),'Interpreter', 'latex');
    legend('$x$','$y$','$z$','Interpreter','latex
        ','location','southwest'); xlabel('Time (
        s)','Interpreter','latex'); ylabel('
        Position (m)','Interpreter','latex');
end
formatFigure('single');
```

```
% plot velocities for each scenario
figure('Name','Velocity traces for scenarios','
    NumberTitle','off');
for si = 1:numel(scenarios)
    % subplot(nRows, nCols, si);
    plot(results(si).t, results(si).X(7,:),
        results(si).t, results(si).X(8,:),
        results(si).t, results(si).X(9,:),'
        LineWidth',1.5);
    grid on;
    title('\textbf{Velocity vs. Time}','
        Interpreter','latex');
    subtitle(sprintf('Scenario %d: %s obs=%d mode
        =%s', results(si).scenario.id, results(si
        ).scenario.type, results(si).scenario.
        useObserver, results(si).scenario.obsMode
        ),'Interpreter', 'latex');
    legend('$v_x$','$v_y$','$v_z$','Interpreter',
        'latex','location','southwest'); xlabel('
        Time (s)','Interpreter','latex'); ylabel(
        'Velocity (m/s)','Interpreter','latex');
end
formatFigure('single');
```

```
% plot motor speeds for each scenario
figure('Name','Motor speeds for scenarios','
    NumberTitle','off');
for si = 1:numel(scenarios)
    % subplot(nRows, nCols, si);
    plot(results(si).t, results(si).Omega(1,:),
        results(si).t, results(si).Omega(2,:),
        results(si).t, results(si).Omega(3,:),
        results(si).t, results(si).Omega(4,:),'
        linewidth',1.5);
    grid on;
    title('\textbf{Motor Speed vs. Time}','
        Interpreter', 'latex');
    subtitle(sprintf('Scenario %d: %s obs=%d mode
        =%s', results(si).scenario.id, results(si
        ).scenario.type, results(si).scenario.
        useObserver, results(si).scenario.obsMode
        ),'Interpreter', 'latex');
    legend('$\omega _1$','$\omega _2$','$\omega
        _3$','$\omega _4$','Interpreter','latex',
        'location','southwest'); xlabel('Time (s)
        ','Interpreter','latex'); ylabel('Speed (
        rpm)','Interpreter','latex');
end
formatFigure('single');
```

3D Plots

```
%% Plot trajectory with waypoints
numScenarios = numel(scenarios);
figure('Name','Quadrotor all trajectories with
    targets','NumberTitle','off');
colors = lines(numScenarios);
hold on;
for si = 1:numScenarios
    X = results(si).X;


    % Plot trajectory
    plot3(X(1,:), X(2,:), X(3,:), 'Color', colors
        (si,:), 'LineWidth', 1.5);
    grid on;

    % Plot target waypoints
    for ti = 1:numel(targets)
        tgt = targets{ti};
        plot3(tgt(1), tgt(2), tgt(3), 'o', '
            MarkerSize', 8, ...
                'MarkerFaceColor', colors(si,:), '
                    MarkerEdgeColor','k');
        text(tgt(1)+0.02, tgt(2)+0.02, tgt(3)
            +0.02, sprintf('T%d', ti)); % label
    end

    xlabel('$x$ (m)','Interpreter', 'latex');
        ylabel('$y$ (m)','Interpreter', 'latex');
         zlabel('$z$ (m)','Interpreter', 'latex')
        ;
```

```
    title('\textbf{3D Trajectory Plot}','
        Interpreter', 'latex')
    subtitle('All Six Scenarios Overlayed','
        Interpreter', 'latex');
    view(260,30);
end
formatFigure('double');
```

```
numScenarios = numel(scenarios);
figure('Name','Quadrotor trajectory with targets'
    ,'NumberTitle','off');
colors = lines(numScenarios);

for si = 1:numScenarios
    X = results(si).X;
    % subplot(nRows, nCols, si);
    hold on; grid on;

    % Plot trajectory
    plot3(X(1,:), X(2,:), X(3,:), 'Color',
        getSColor(results(si).scenario.id), '
        LineWidth', 1.5);
    grid on;

    % Plot target waypoints
    for ti = 1:numel(targets)
        tgt = targets{ti};
        plot3(tgt(1), tgt(2), tgt(3), 'o', '
            MarkerSize', 8, ...
              'MarkerFaceColor', getSColor(
                  results(si).scenario.id), '
                  MarkerEdgeColor','k');
        text(tgt(1)+0.02, tgt(2)+0.02, tgt(3)
            +0.02, sprintf('T%d', ti)); % label
    end

    xlabel('$x$ (m)','Interpreter', 'latex');
        ylabel('$y$ (m)','Interpreter', 'latex');
         zlabel('$z$ (m)','Interpreter', 'latex')
        ;
    xlabel('$x$ (m)','Interpreter', 'latex');
        ylabel('$y$ (m)','Interpreter', 'latex');
         zlabel('$z$ (m)','Interpreter', 'latex')
        ;
    title('\textbf{3D Trajectory Plot}','
        Interpreter', 'latex')
    subtitle(sprintf('Scenario %d: %s obs=%d mode
        =%s', ...
          results(si).scenario.id, results(si).
              scenario.type, ...
          results(si).scenario.useObserver, ...
          results(si).scenario.obsMode),'
              Interpreter', 'latex');
    view(260,30);
end
formatFigure('single');
```

```
% used to manually export 3D plots with high
    resolution
exportgraphics(gca, 'sALL.pdf', 'Resolution',
    600);
```

Observer Scenario State Estimation Error

```
%       Estimation Error Plots for Observer
    Scenarios

figure('Name','Estimation Error for Observer
    Scenarios','NumberTitle','off');
hold on; grid on;
colors = lines(numel(scenarios));

for si = 1:numel(scenarios)
    sc = scenarios{si};

    % Skip scenarios that did NOT use an observer
    if ~sc.useObserver
        continue;
    end

    X       = results(si).X;       % true state
    Xhat    = results(si).Xhat;    % estimated
        state
    t       = results(si).t;       % time vector

    % Compute estimation error norm at each
        timestep
    e       = vecnorm(X - Xhat, 2, 1);   % 1xN
        vector

    % Plot
    plot(t, e, 'LineWidth', 1.5, ...
         'DisplayName', sprintf('Scenario %d (%s,
             %s observer)', ...
                    sc.id, sc.type, sc.obsMode))
                        ;
end

xlabel('Time (s)','Interpreter','latex');
ylabel('Estimation Error  $\vert\vert \,x - \hat{
    x}\,\vert\vert$','Interpreter','latex');
title('\textbf{State Estimation Error vs. Time}',
    'Interpreter','latex');
legend('Location','southoutside','Interpreter','
    latex','FontSize',7.5);
formatFigure('double');
```

Position and Velocity Metric Analysis

```
%% Threshold definitions
posThreshold     = 0.10;   % [m]
velPercent       = 0.10;   % relative speed error
velZeroThreshold = 0.10;   % [m/s]
velAngleDeg      = 10;     % [deg]
settleThreshold  = 0.10;   % [m] settling band

numScenarios = numel(scenarios);

for si = 1:numScenarios

    X = results(si).X;
    t = results(si).t;
    N = size(X,2);

    %% POSITION METRICS PER WAYPOINT
    numWP = numel(targets);

    Pos_wp           = zeros(numWP,3);
    Pos_ach          = zeros(numWP,3);
    Pos_minDist      = zeros(numWP,1);
    Pos_reached      = strings(numWP,1);
    Pos_orderOK      = strings(numWP,1);

    rmsPos           = zeros(numWP,1);
    overshoot        = zeros(numWP,1);
    tSettle_num      = nan(numWP,1);
    tSettle_status   = strings(numWP,1);

    settleThreshold = posThreshold;   % 0.10 m
        band for settling

    lastIdx = 0;   % last "closest index" for
        previous waypoint
```

```
    for ti = 1:numWP

        tgt_pos   = targets{ti}(1:3);
        distances = vecnorm(X(1:3,:) - tgt_pos,
            2, 1);   % 1 x N

        % define search region for this waypoint
            (after previous waypoint)
        searchIdx = (lastIdx+1):N;
        if isempty(searchIdx)
            % no future samples, record and
                continue
            Pos_wp(ti,:)        = tgt_pos.';
            Pos_ach(ti,:)       = [NaN NaN NaN];
            Pos_minDist(ti)     = NaN;
            Pos_reached(ti)     = "false";
            Pos_orderOK(ti)     = "false";
            rmsPos(ti)          = NaN;
            overshoot(ti)       = NaN;
            tSettle_num(ti)     = NaN;
            tSettle_status(ti)  = "
                no_future_samples";
            continue;
        end

        % global closest approach within the
            search region
        [minDist, relIdx] = min(distances(
            searchIdx));
        idxMin             = searchIdx(relIdx);
                % global index of min
        pos_at_wp          = X(1:3, idxMin);

        reached = (minDist < posThreshold);
        orderOk = reached;   % if it never comes
            close, we call order violated

        if reached
            lastIdx = idxMin;   % advance for
                next waypoint
        end

        % define SEGMENT: from search start to
            global closest approach
        segStartIdx = searchIdx(1);
        segEndIdx   = idxMin;
        segIdx      = segStartIdx:segEndIdx;

        % RMS position error on this segment
        posErrSeg    = X(1:3, segIdx) - tgt_pos;
        posErrMagSeg = vecnorm(posErrSeg, 2, 1);
        rmsPos_k     = sqrt(mean(posErrMagSeg.^2)
            );

        % Overshoot: max distance AFTER closest
            approach minus minDist
        if idxMin < searchIdx(end)
            afterIdx     = idxMin:searchIdx(end);
            maxDistAfter = max(distances(afterIdx
                ));
            overshoot_k  = max(0, maxDistAfter -
                minDist);
        else
            overshoot_k = 0;   % no samples after
                 closest point
        end

        % Settling time: first time we enter 0.1
            m band ON THIS SEGMENT
        loc = find(distances(segIdx) <=
            settleThreshold, 1, 'first');

        if isempty(loc)
            % never entered band on this segment
            tSettle_num(ti)    = NaN;
            tSettle_status(ti) = "
                never_entered_band";
        else
            settleIdx          = segIdx(loc);
                        % global index of first
                entry
            tSettle_num(ti)    = t(settleIdx) - t
                (segStartIdx);
            tSettle_status(ti) = "
                settled_first_entry";
        end

        % store row
        Pos_wp(ti,:)      = tgt_pos.';
        Pos_ach(ti,:)     = pos_at_wp.';
        Pos_minDist(ti)   = minDist;
        Pos_reached(ti)   = string(reached);
        Pos_orderOK(ti)   = string(orderOk);
        rmsPos(ti)        = rmsPos_k;
        overshoot(ti)     = overshoot_k;

    end

    PositionTable = table( ...
        Pos_wp, Pos_ach, Pos_minDist, Pos_reached
            , ...
        rmsPos, overshoot, tSettle_num, ...
        'VariableNames', {'posDesired','pos_at_wp
            ','minDist','posReached', ...
                          'rmsPos','overshoot','
                              tSettle'} ...
    );

    fprintf('\nPOSITION RESULTS, SCENARIO %d\n',
        results(si).scenario.id);
    disp(PositionTable);



    %% VELOCITY CHECK (per scenario)
    Vel_wp       = zeros(numel(targets),3);
    Vel_ach      = zeros(numel(targets),3);
    Vel_speedErr = zeros(numel(targets),1);
    Vel_angDeg   = zeros(numel(targets),1);
    Vel_status   = strings(numel(targets),1);

    lastIdx = 0;   % reset for velocity for this
        scenario

    for ti = 1:numel(targets)

        tgt_pos = targets{ti}(1:3);
        tgt_vel = targets{ti}(7:9);
        targetSpeed = norm(tgt_vel);

        distances = vecnorm(X(1:3,:) - tgt_pos,
            2, 1);
        searchIdx = (lastIdx+1):N;
        nearIdx   = find(distances(searchIdx) <
            posThreshold);
        nearIdx   = searchIdx(nearIdx);

        Vel_wp(ti,:) = tgt_vel.';

        % CASE 1: waypoint not reached
        if isempty(nearIdx)

            [~, relIdx] = min(distances(searchIdx
                ));
            bestIdx = searchIdx(relIdx);

            v_at_wp = X(7:9, bestIdx);
            actSpeed = norm(v_at_wp);
```

```
        speedErr = abs(actSpeed - targetSpeed
            );

        if targetSpeed < 1e-3
            angDeg = NaN;
        elseif actSpeed < 1e-3
            angDeg = 180;
        else
            tgt_dir = tgt_vel / targetSpeed;
            cosang  = dot(v_at_wp, tgt_dir) /
                 (actSpeed * targetSpeed);
            angDeg  = acosd(max(min(cosang,1)
                ,-1));
        end

        Vel_ach(ti,:)       = v_at_wp.';
        Vel_speedErr(ti)    = speedErr;
        Vel_angDeg(ti)      = angDeg;
        Vel_status(ti)      = "CLOSEST (
            waypoint not reached)";

        continue
    end

    % CASE 2: proper window
    idxStart = nearIdx(1);
    validIdx = idxStart;
    for k = idxStart+1:N
        if distances(k) > posThreshold, break
            ; end
        validIdx(end+1) = k;
    end

    v_candidates = X(7:9, validIdx);
    actSpeeds    = vecnorm(v_candidates,2,1);

    if targetSpeed < 1e-3
        [minSpeed, idxMinSpeed] = min(
            actSpeeds);
        v_at_wp = v_candidates(:, idxMinSpeed
            );
        speedErr = minSpeed;
        angDeg   = NaN;

        if speedErr < velZeroThreshold
            statusTxt = "OK (hover speed)";
        else
            statusTxt = "speed error";
        end

    else
        tgt_dir = tgt_vel / targetSpeed;
        angErrs   = zeros(1, numel(validIdx))
            ;
        speedErrs = zeros(1, numel(validIdx))
            ;

        for k = 1:numel(validIdx)
            actSpeed = actSpeeds(k);
            if actSpeed < 1e-3
                angErrs(k) = 180;
            else
                cosang = dot(v_candidates(:,k
                    ), tgt_dir)/(actSpeed*
                    targetSpeed);
                cosang = max(min(cosang,1),
                    -1);
                angErrs(k) = acosd(cosang);
            end
            speedErrs(k) = abs(actSpeed -
                targetSpeed);
        end

        cost = speedErrs + (angErrs/180)*
                targetSpeed;
            [~, bestIdx] = min(cost);

            v_at_wp = v_candidates(:, bestIdx);
            speedErr = speedErrs(bestIdx);
            angDeg   = angErrs(bestIdx);

            dirOK = angDeg < velAngleDeg;
            magOK = speedErr < velPercent *
                targetSpeed;

            if     dirOK && magOK, statusTxt = "
                angle & speed OK";
            elseif ~dirOK && magOK, statusTxt = "
                angle error";
            elseif dirOK && ~magOK, statusTxt = "
                speed error";
            else                    statusTxt = "
                angle & speed error";
            end
        end

        lastIdx = validIdx(end);

        Vel_ach(ti,:)    = v_at_wp.';
        Vel_speedErr(ti) = speedErr;
        Vel_angDeg(ti)   = angDeg;
        Vel_status(ti)   = statusTxt;

    end

    VelocityTable = table( ...
        Vel_wp, Vel_ach, Vel_speedErr, Vel_angDeg
            , Vel_status, ...
        'VariableNames', {'velDesired','v_at_wp',
            'speedErr','angDeg','status'} ...
    );

    fprintf('VELOCITY RESULTS, SCENARIO %d\n',
        results(si).scenario.id);
    disp(VelocityTable);

end
```

User-Defined Functions

```
function formatFigure(which, exportName)
%FORMATFIGURE   Format the current figure for 1-
    or 2-column LaTeX reports.
%
%   Inputs:
%       which       'single' or 'double' column
    width
%       exportName  (optional) filename to export
     as PDF
%
%   This function:
%       sizes a figure for single- or double-
    column IEEE layout

    if nargin < 1
        which = 'single';
    end

    fig = gcf;
    set(fig, 'Units', 'inches');

    switch lower(which)
        case 'single'
            width  = 3.3;
            height = 2.2;
        case 'double'
            width  = 6.8;   % span two columns
```

```
            height = 2.5;
        otherwise
            error('formatFigure:InvalidOption',
                ...
                'Choose either ''single'' or ''
                    double''.');
    end

    % Figure sizing
    set(fig, 'Position', [1 1 width height]);

    % Axes and line formatting
    ax = gca;
    set(ax, 'FontSize', 8);

    % Set line widths
    lines = findall(fig, 'Type', 'line');
    set(lines, 'LineWidth', 1.0);

    % If quiver/patch objects exist, format them
        too
    patches = findall(fig, 'Type', 'patch');
    set(patches, 'LineWidth', 1.0);

    % export, I ended up just doing this
        separately
    if nargin == 2 && ~isempty(exportName)
        fprintf('Exporting figure to: %s\n',
            exportName);
        exportgraphics(fig, exportName, '
            ContentType', 'vector');
    end

end

function c = getSColor(id)
%getScolor color assignment for 3D scenario
    plotting
%
%   c = getSColor(id)
%
%   id : integer scenario index (1, 2, 3, ...)
%   c  : 1x3 RGB color row vector in [0,1]

    palette = [
        0.00 0.45 0.74;   % Scenario 1 - blue
        0.85 0.33 0.10;   % Scenario 2 - red/
            orange
        0.47 0.67 0.19;   % Scenario 3 - green
        0.93 0.69 0.13;   % Scenario 4 - yellow/
            gold
        0.49 0.18 0.56;   % Scenario 5 - purple
        0.30 0.75 0.93;   % Scenario 6 - cyan
        0.64 0.08 0.18;   % realistic Cstruct -
            dark red
    ];

    N = size(palette,1);
    index = mod(id-1, N) + 1;

    c = palette(index,:);
end
```